

Step 4 — Graceful Degradation When One MCP Server Is Down — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real timeout/try-except shapes), **Advanced** (what it actually takes for the *whole team*, not just one function, to degrade gracefully). Read Basic first even if you already built Project 8's Step 4 — it's the fastest way to spot exactly what each deeper level adds at the team level.

- [Hint 1](#) — Where the failure has to be caught, and where it must not leak
- [Hint 2](#) — The plan, and almost the whole thing

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Where the failure has to be caught, and where it must not leak

Basic Version

If `web_server.py` was never started, connecting to it will either raise an exception quickly, or hang. Either way, something needs to catch that, right where the connection is attempted, and turn it into a normal piece of data (a flag, a string) instead of letting it crash the whole graph run.

Things to use: - `asyncio.wait_for(..., timeout=...)` around the connect-and-initialize step. - `try / except` around that, inside `web_agent.py` itself — not inside `supervisor.py`, and not inside `graph.py`. - A new state field that says plainly "the web check couldn't happen."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The failure needs to be caught at the exact layer that's actually calling the connection —

`_run_web_agent` (or wherever your `connect_stdio_server(...)` call lives). If you catch it any higher up (inside `web_agent_node`, or worse, inside `supervisor.py`), you've mixed "how do I talk to my own server" concerns into code that shouldn't need to know MCP exists at all — remember Project 4's original design lesson: the Supervisor never touches a specialist's own connection details directly, it only reads that specialist's *result*.

The exact pieces: - `asyncio.wait_for(_connect_and_run(...), timeout=10)` wrapping the whole connect-through-answer sequence, not just the `initialize()` call — a hang could happen anywhere in that sequence, not only during the handshake. - `except (asyncio.TimeoutError, Exception) as e:` — broad on purpose here, the same way Project 8 Step 4 caught broadly at this one specific boundary, because *any* failure at this layer means the same thing: "this server isn't usable right now." - `run_web_agent` returns a normal string either way — either the real answer, or a sentinel like `"WEB_UNAVAILABLE: <reason>"` — so every caller above it keeps working with a plain string, and never needs its own try/except for this. - `web_agent_node` checks for that sentinel prefix and sets `state["web_unavailable"] = True` when it sees it, instead of putting the raw sentinel string into `web_findings`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Trace the whole path a degraded run needs to take, node by node, before you write any code:

Supervisor routes to `web_agent` (it doesn't know or care that the server is down) →

`web_agent_node` calls `run_web_agent`, which fails internally and returns the sentinel →

`web_agent_node` catches the sentinel, sets `web_unavailable=True`, still marks itself as visited (this matters — without marking `visited`, the Supervisor would try `web_agent` again and again, forever) → routes back to Supervisor → Supervisor sees `web_agent` is visited, and now also needs to route to `writer_agent` even though `web_findings` is empty and `web_unavailable` is `True` — the Supervisor's "is everything needed done" check must not require `web_findings` to be non-empty, only that `web_agent` has been visited.

This is the exact same "did this actually succeed, or did it just not throw an exception" question Doc11's Core Concepts raises about error propagation between agents — a specialist that "fails gracefully" by returning an empty string that looks like a completed-but-empty result is actually worse than one that clearly flags itself as unavailable, because the Supervisor (and the Writer) can't tell "checked, found nothing" from "never actually checked" unless something tells them explicitly.

Also think about what the Notes Agent should do in this same scenario. It shouldn't be affected at all — it has its own separate connection to its own separate server, and Step 1's whole point was proving that isolation. If a bug in your degradation code somehow makes the Notes Agent fail too when only the web server is down, that's a sign something got shared between the two specialists that shouldn't have been (the same warning Step 1's Advanced hint gave about not sharing one connection between unrelated specialists).

Things to try before Hint 2: - Run the full team with `web_server.py` genuinely not started (don't launch it in a separate terminal, don't fake the failure in code) — confirm you see a real timeout or connection error caught, not a hang, and not a crash. - Run it again immediately after, with `web_server.py` started normally — confirm nothing about your Step 3 behavior changed for the working case.

Difference between Basic, Intermediate, and Advanced: Basic says where to catch the failure and what to catch it with. Intermediate gives the exact `asyncio.wait_for / try / except / sentinel` shape, and where it does and doesn't belong. Advanced traces the full path a degraded run has to take through every node, including the two easy-to-miss bugs (an infinite retry loop if `visited` isn't marked, and a Supervisor that waits forever for a non-empty `web_findings` that will never come) that make "it degrades gracefully" actually true end to end, not just true for the one function that caught the exception.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
run_web_agent(task):
    try, with a timeout:
        connect, discover tools, run the loop, return the real answer
    except timeout or any error:
        return "WEB_UNAVAILABLE: <what went wrong>"

web_agent_node(state):
    result = run_web_agent(state["task"])
    if result starts with "WEB_UNAVAILABLE":
        mark web_unavailable = True in state, web_findings stays empty
    else:
        save result into web_findings
    mark web_agent as visited either way
    route back to supervisor

supervisor_node(state): same as Step 3, but "done with a specialist" only
checks visited, never checks whether its findings are non-empty

writer_agent's prompt: if web_unavailable is True, say plainly the web
check could not be completed
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# web_agent.py (relevant part)
import asyncio

WEB_UNAVAILABLE_PREFIX = "WEB_UNAVAILABLE"

async def _run_web_agent(task: str, max_iterations: int = 5) -> str:
    # ... same body as Step 1/2/3 ...
    ...

def run_web_agent(task: str, timeout: float = 10.0) -> str:
    try:
        return asyncio.run(asyncio.wait_for(_run_web_agent(task), timeout=timeout))
    except asyncio.TimeoutError:
        return f"{WEB_UNAVAILABLE_PREFIX}: timed out after {timeout}s connecting"
    except Exception as e:
        return f"{WEB_UNAVAILABLE_PREFIX}: {type(e).__name__}: {e}"
```

Your turn: update `web_agent_node` to check for the `WEB_UNAVAILABLE` prefix and set `state["web_unavailable"]`, update `supervisor_node` so its "done" check only looks at `visited`, and update the Writer's prompt to read `web_unavailable`. Compare all of it against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in the node and Writer changes yourself, using this skeleton:

```
# web_agent.py (node function, updated)
from langgraph.types import Command
from state import TeamState
# run_web_agent(task) and WEB_UNAVAILABLE_PREFIX already exist above

def web_agent_node(state: TeamState) -> Command:
    result = run_web_agent(state["task"])
    visited = state.get("visited", []) + ["web_agent"]

    if result.startswith(WEB_UNAVAILABLE_PREFIX):
        # your turn: log this clearly, and update state with
        # web_unavailable=True instead of writing the raw sentinel
        # into web_findings
        ...

    return Command(goto="supervisor", update={"web_findings": result, "visited":
```

```
# writer_agent.py (prompt update)
WRITER_PROMPT = """You are writing a short status report from research findings.
```

```
Notes findings:
{notes_findings}
```

```
Web findings:
{web_findings}
```

```
Write one short, clear report. State what is known, citing which source(s)
support each point. Do not write one paragraph per source -- organize by
topic or conclusion instead.{degradation_note}"""
```

```
def run_writer(notes_findings: str, web_findings: str, web_unavailable: bool = Fa
    # your turn: build a degradation_note string that's empty when
    # web_unavailable is False, and a clear instruction to say the web
    # check failed when it's True -- then format it into WRITER_PROMPT
    ...
```

Run task 3 with `web_server.py` never started, and confirm the final report explicitly says the web check couldn't be completed, before comparing against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get the failure caught and flagged correctly inside `web_agent.py`. Advanced is where that flag actually has to travel all the way to the one place a human reads — the Writer's final report — which means touching `writer_agent.py`'s prompt-building, not just `web_agent.py`. A flag that's set correctly in state but never reaches the report isn't graceful degradation yet, it's just a silent internal detail.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)